PHILIPS MR TEAM · AGENTIC AI WORKSHOP · 4–5 NOVEMBER 2026

Agentic AI Workshop: Spec, Build, Guard, Break

Spec-driven development in VS Code with Copilot / Codex, a multi-agent app with MCP, then guardrails, eval harness and red-teaming on AWS Bedrock

CLIENT

Philips MR Team

DATES

4–5 Nov 2026 · 2 days

CLOUD

AWS Bedrock + Guardrails

FORMAT

In-person · laptops · hands-on



SPEC

Spec → Plan → Tasks



BUILD

Agents + MCP + UI



GUARD

Bedrock Guardrails + harness



BREAK

Red-team & harden

unext

WHAT THE TEAM LEAVES WITH

Four Artefacts Every Pod Takes Home

Built hands-on in the room, committed to each pod's repo, reusable the next working day.

Executable spec

spec.md, plan.md and tasks.md that Copilot / Codex can execute — intent, scope, data, tools, guardrails, acceptance tests.

Working agentic app

FastAPI service on the 10-point scaffold, LangGraph orchestrator with domain agents, an MCP agent, a database and a Streamlit UI.

Guarded + evaluated

Bedrock Guardrails plus four app-level guard layers, and an eval harness that runs as a CI gate on every change.

Red-team report

Attack scripts, a log of every break, the fix for each, and a resilience scorecard against agreed thresholds.

2 days, in person

1 agentic use case, end to end

20 OWASP risks mapped to the app

~70% hands-on lab time

Laptop, Coding Agent, AWS Bedrock — How It Fits

Coding assistance uses Philips-provided endpoints; the app's own model calls and guardrails run in the Philips AWS account.

Participant laptop

- VS Code + Python 3.11+
- GitHub Copilot (agent mode) or OpenAI Codex
- Spec files, AGENTS.md, Skills
- Git repo per pod

Agentic app (runs locally)

- Streamlit UI
- FastAPI service (10-point scaffold)
- LangGraph orchestrator + agents
- MCP server + SQLite / Postgres

AWS — Philips account

- Amazon Bedrock models (Converse API)
- Bedrock Guardrails (ApplyGuardrail)
- IAM / SSO, approved region
- CloudWatch logs for audit

Data rule for the room: synthetic MR operational data only — no PHI, no patient data, no production systems. Coding endpoints: Philips-provided OpenAI / Claude API.

Day 1 Builds It; Day 2 Guards It and Breaks It

DAY 1 · Wed 4 Nov — Spec-Driven Build	
09:00–09:30	Kickoff + environment check
09:30–11:00	S1 · Spec-driven development: Spec ⟶ Plan ⟶ Tasks ⟶ Code
11:15–13:00	S2 · Lab: write the use-case spec; generate the scaffold
14:00–14:45	S3 · Agent orchestration, MCP concepts, LangChain · LangGraph · CrewAI
14:45–15:45	S4a · Build: agents, MCP server and MCP agent
16:00–17:15	S4b · Build: Skills, database, Streamlit UI, end-to-end run
17:15–17:30	Demo round + commit day1-done

DAY 2 · Thu 5 Nov — Guard, Evaluate, Break	
09:00–09:30	Day 1 recap · why guardrails
09:30–11:00	S5 · OWASP LLM Top 10 + Agentic Top 10, mapped to our app
11:15–13:00	S6 · Lab: Bedrock Guardrails, app guard layers, eval harness
14:00–15:30	S7 · Red-team: break the Day 1 app with attack scripts
15:45–16:45	S8 · Fix, re-run the harness, resilience scorecard
16:45–17:30	Showcase, takeaways and close

Breaks: 11:00–11:15 tea · 13:00–14:00 lunch · afternoon tea 15:45 (Day 1) / 15:30 (Day 2). Pods of 3 with rotating roles: spec owner · builder · reviewer.

DAY 1 · WEDNESDAY 4 NOVEMBER

Spec-Driven Build

Environment check and SDD in the morning. Orchestration, MCP and a working multi-agent app — FastAPI, Skills, database, Streamlit — in the afternoon.

unext

Six Green Ticks Before We Write a Line

Every pod confirms all six. The trainer validates the same checks on 3 November with Philips-provided access.

Toolchain

VS Code, Python 3.11+, Git, uv or pip; repo cloned and virtual env created.

Coding agent

Copilot agent mode or Codex signed in and answering via the Philips endpoint.

AWS identity

SSO / IAM credentials work: aws sts get-caller-identity returns the workshop role.

Bedrock model call

Smoke test: one Converse API call to the approved model in the approved region.

Guardrail call

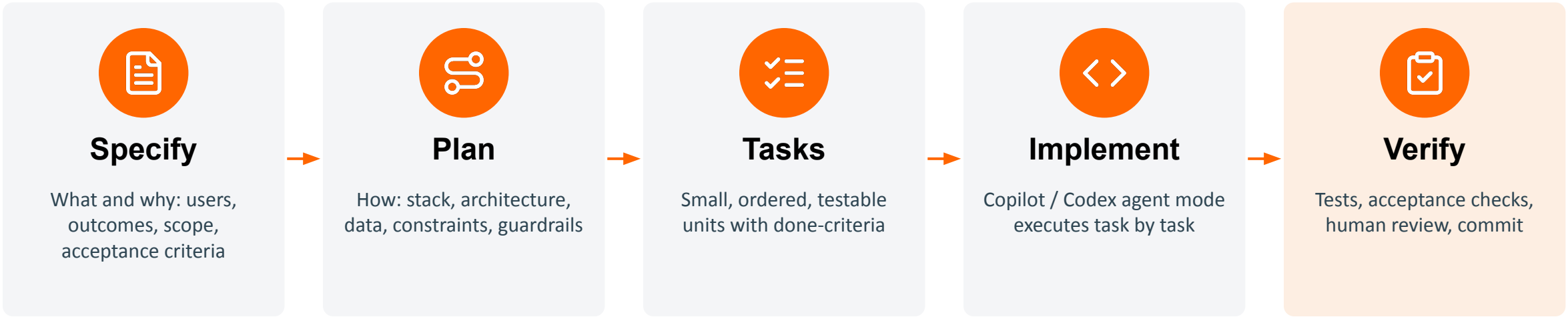
Smoke test: ApplyGuardrail on a sample prompt returns an intervention.

Starter repo

make check passes: lint, type check, unit tests green on the empty scaffold.

Spec-Driven Development: The Spec Is the Truth

Prompts are disposable; specs are versioned. The coding agent executes the spec — it does not guess intent.



Prompt-driven (vibe coding)

- Intent lives in chat history and is lost
- Output drifts between runs and developers
- Review happens after the code exists

Spec-driven

- Intent is a versioned file the agent reads every run
- Regenerate or extend from the same spec
- Review the spec and plan before code is written

Anatomy of a Spec an Agent Can Execute

Each pod writes this for the Day 1 use case, then has Copilot or Codex generate plan, tasks and the scaffold from it.

1	**Intent and users**	Who uses it, what job it does, what success looks like
2	**Scope and non-goals**	What the agent must never do — refusals belong here
3	**Data and classification**	Entities, sources, PUBLIC / INTERNAL / CONFIDENTIAL tags
4	**Agents and MCP tools**	Roles, handoffs, tool list with argument schemas
5	**Guardrails**	Input, tool-call, output and approval rules, declared up front
6	**Acceptance tests**	Given / when / then cases the eval harness will run
7	**Non-functionals**	Latency, cost ceiling, audit log, the 10-point scaffold

Pod repo layout

```
specs/001-mr-service-agent/
   spec.md · plan.md · tasks.md
AGENTS.md              ← rules for Codex
.github/copilot-instructions.md
.github/prompts/       ← reusable prompts
skills/                ← SKILL.md packs
app/  core/  adapters/  mcp_server/
tests/  evals/  redteam/
```

MR Service Insight Agent — One Use Case, End to End

Proposed default, to be confirmed with the Philips MR team. Any replacement keeps the same architecture and labs.



Ticket in

Synthetic MR service ticket: error code, logs, site

Triage agent

Classifies fault area, sets priority

Knowledge agent

Finds similar cases and SOPs via MCP tools

Resolution agent

Drafts an action plan with cited sources

Human gate

Engineer approves before the ticket is updated

MCP tools

search_tickets · get_error_code · find_similar_cases · update_ticket_status (guarded)

Data

SQLite (Postgres optional) for tickets; vector index for SOPs and past cases

UI

Streamlit: ticket queue, agent trace, approve / reject

Alternates

CAPA / change-request summariser · test-log analyser · release-note writer

Five Orchestration Patterns and When to Use Them

Sequential

Fixed pipeline; each step feeds the next. Predictable, cheap, easy to test.

Supervisor

Orchestrator routes work to specialist agents and merges results. Our Day 1 build.

Router / handoff

Classify once, hand control to the right agent with typed state.

Parallel fan-out

Independent sub-tasks run concurrently, then aggregate.

Evaluator loop

Generator plus critic iterate until a rubric passes or a limit is hit.

Failure modes we design against

Infinite loops

Fix: recursion limit + step budget

Context bloat

Fix: typed state, not raw transcripts

Cascading errors

Fix: validate every handoff

Lost work

Fix: checkpoints and resumable runs

LangChain, LangGraph, CrewAI — Which Does What

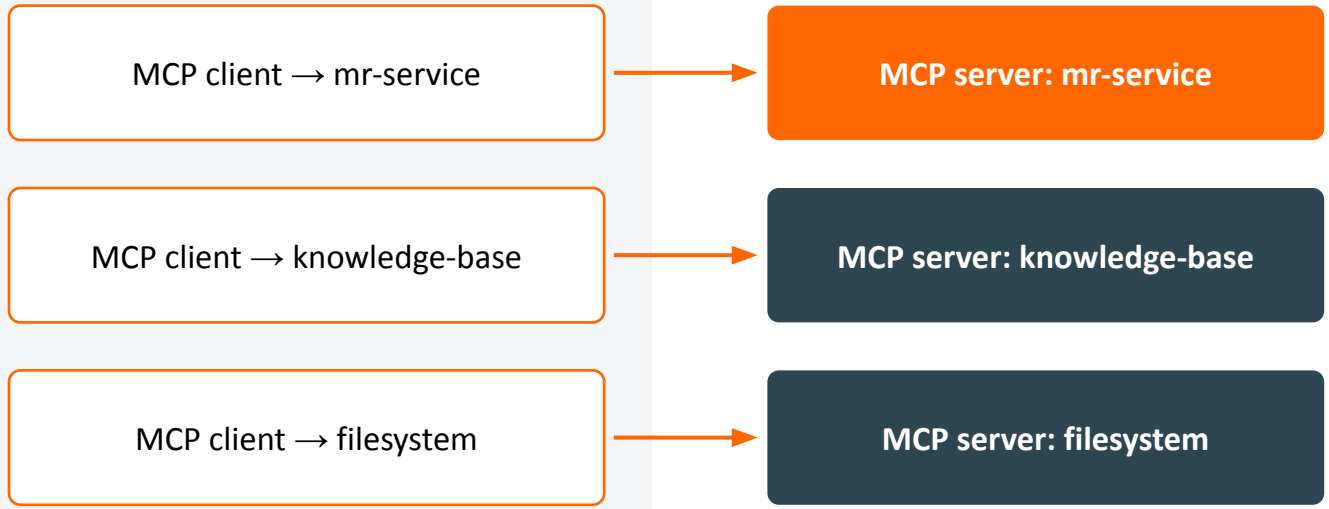
	LangChain	LangGraph	CrewAI
Mental model	Building blocks: models, tools, prompts, retrievers, create_agent	Stateful graph: nodes, edges, typed shared state	Role-based crew: agents, tasks, sequential or hierarchical process
Best at	Single agent with tool calling; model and tool wrappers	Production orchestration with loops, branches and retries	Fast prototyping of a team of specialist roles
State and HITL	Middleware and memory; basic	Checkpointing, interrupts for human approval, time-travel	Crew memory; human input on tasks; Flows for control
Bedrock	ChatBedrockConverse (langchain-aws)	Same model objects as LangChain	LLM configured with a bedrock/ model id
In this workshop	Bedrock model + tool wrappers	**Main orchestrator for the Day 1 app**	Stretch: rebuild the same flow and compare

Rule of thumb: start with LangGraph when you need control and auditability; use CrewAI to explore role designs quickly; LangChain supplies the parts for both.

MCP: Tools Agents Can Discover and Call

MCP HOST

Our agent app, or VS Code Copilot



stdio locally · Streamable HTTP when remote

Primitives

Tools (actions), Resources (read-only context), Prompts (templates)

What we build

mr-service server with FastMCP: 4 typed tools, tested in MCP Inspector, then consumed by our MCP agent and by Copilot

Security

Least-privilege tools, argument schemas, auth on remote servers, allowlisted servers only — tool descriptions are untrusted input

The 10-Point FastAPI Scaffold Every Pod Builds On

Treat the model as a flaky vendor. The harness is what you ship — and what lets a coding agent change code safely.

1. **Type hints on every boundary**
   Wrong types rejected where they enter
2. **Pydantic v2 strict models**
   All LLM output parsed, never raw dicts
3. **Async I/O throughout**
   No blocking calls inside the event loop
4. **Typed client, retries + backoff**
   Timeout, jitter, capped attempts, fallback
5. **Provider behind a port**
   Bedrock is an adapter, swappable in one file
6. **Tool args validated first**
   Schema check before any tool code runs
7. **Guards on destructive tools**
   Business rules, authorisation, human approval
8. **Request and response models**
   Bad data kept out, secrets kept in
9. **Contract tests as a CI gate**
   Assert shape and properties, not wording
10. **Pure core**
    No SDK, SQL or framework imports in the domain

How the Day 1 App Is Layered

Streamlit UI — Ticket queue · agent trace · approve / reject

↓

FastAPI airlock — Request + response models · auth · /triage /approve /health

↓

LangGraph orchestrator — Supervisor ⟶ triage, knowledge, resolution agents · checkpoints · interrupt for human gate

↓

Pure core — Use cases, domain models, ports — no SDK, SQL or framework imports

↓

LLM port	**Tool port**	**Repo port**
Bedrock adapter (Converse)	MCP client ⟶ mr-service	SQLite / Postgres + vector index

Guard slots, empty on Day 1

- L1 input pre-hook — before the orchestrator
- L2 tool-call guard — inside the tool port
- L3 output validator — before the response model
- L4 audit + human gate — at the interrupt

Day 2 fills each slot — no re-architecture needed.

Skills Teach the Coding Agent How Your Team Builds

A skill is a folder with a SKILL.md and optional scripts and references, loaded on demand by the coding agent.

Skills vs MCP — not the same thing

Skills
Shape how the coding agent writes code: conventions, checklists, templates, scripts. Used at build time.

MCP tools
Actions the runtime agent calls while serving a user: search tickets, update status. Used at run time.

Skills run in Copilot agent mode and Codex; confirmed in the environment check.

```
---
name: fastapi-ai-scaffolding
description: Use when building a FastAPI
  service that calls an LLM ...
---
```

`fastapi-ai-scaffolding`
Builds and audits the 10-point scaffold — Day 1

`mcp-tool-builder`
Typed FastMCP tools with schemas and guards — Day 1

`guardrail-hooks`
Wires Bedrock Guardrails and L1–L4 hooks — Day 2

`redteam-runner`
Runs attack suites and writes the break log — Day 2

Six Checkpoints From Spec to Running App

CP1	CP2	CP3	CP4	CP5	CP6
Scaffold	**Bedrock adapter**	**Agents**	**MCP**	**DB + UI**	**Demo**
Generated from spec; make check green	Typed client, retries, strict parsing	LangGraph supervisor + 3 agents	mr-service server + MCP agent wired	Tickets in SQLite; Streamlit front end	End-to-end run; tag day1-done

How the lab runs

- Coding agent executes tasks.md one task at a time
- Reviewer approves each diff before the next task
- Trainer gate at every checkpoint; fallback branch per CP

Stretch goals for fast pods

- Rebuild the flow in CrewAI and compare traces
- Expose mr-service to Copilot in VS Code
- Add a fourth agent from a new spec section

DAY 2 · THURSDAY 5 NOVEMBER

Guard, Evaluate, Break

Why guardrails and eval harnesses matter, OWASP LLM and Agentic Top 10s, Bedrock Guardrails in the app — then red-team the Day 1 build and harden it.

unext

Why Guardrails and an Eval Harness Are Not Optional

Why guardrails

- Agents act: a bad output becomes a bad database write
- Every ticket, log or SOP is untrusted input to the model
- Regulated MR context: data boundaries must be provable
- Model-side safety alone does not know your business rules

Why an eval harness

- Model output varies run to run; unit tests alone flap
- Model, prompt or guardrail changes silently shift quality
- A fixed golden set turns “seems fine” into a number
- As a CI gate it blocks regressions before they ship

Guardrails stop known-bad behaviour at runtime. The harness proves, on every change, that they still do — and that the app still does its job.

OWASP Top 10 for LLM Applications (2025)

LLM01 — **Prompt Injection**
Instructions hidden in a ticket body

LLM02 — **Sensitive Info Disclosure**
Site or engineer details leak in answers

LLM03 — **Supply Chain**
Unpinned packages, third-party MCP servers

LLM04 — **Data and Model Poisoning**
Poisoned SOPs in the knowledge base

LLM05 — **Improper Output Handling**
Model output rendered or executed unchecked

LLM06 — **Excessive Agency**
Agent closes tickets without approval

LLM07 — **System Prompt Leakage**
Prompt reveals rules, keys or endpoints

LLM08 — **Vector and Embedding Weaknesses**
Retrieval crosses data classifications

LLM09 — **Misinformation**
Confident fix with no cited source

LLM10 — **Unbounded Consumption**
Agent loops burn tokens and cost

OWASP Top 10 for Agentic Applications (2026)

ASI01 — **Agent Goal Hijack**
Ticket text redirects the supervisor's goal

ASI02 — **Tool Misuse and Exploitation**
update_ticket_status called with a broad filter

ASI03 — **Identity and Privilege Abuse**
Agent acts with the developer's AWS role

ASI04 — **Agentic Supply Chain**
Unvetted MCP server or tool description

ASI05 — **Unexpected Code Execution**
Generated code or shell run without a sandbox

ASI06 — **Memory and Context Poisoning**
Bad case written to memory skews later runs

ASI07 — **Insecure Inter-Agent Comms**
Untyped handoffs between our three agents

ASI08 — **Cascading Failures**
Triage error propagates to resolution and DB

ASI09 — **Human-Agent Trust Exploitation**
Polished draft rubber-stamped at the human gate

ASI10 — **Rogue Agents**
Agent drifts from its spec over many runs

Defence in Depth: Bedrock Guardrails + App Layers

Amazon Bedrock Guardrails

Content filters — incl. prompt-attack detection

Denied topics — e.g. clinical diagnosis advice

Word filters — profanity and custom terms

Sensitive information filters — PII types plus custom regex

Contextual grounding check — answers tied to retrieved SOPs

Automated Reasoning checks — policy rules checked formally

ApplyGuardrail API screens text for any model — including the Philips OpenAI / Claude endpoints.

L1 — Input pre-hook

Guardrail on user and ticket text; data-class check before the orchestrator

L2 — Tool-call guard

Schema, allowlist, business rule, auth and call budget on every MCP call

L3 — Output validator

Guardrail on output, strict model parse, PII scan, citation required

L4 — Audit + human gate

Structured log per step; LangGraph interrupt for status changes

An Eval Harness That Gates Every Change

Golden set

30–50 cases from the spec's acceptance tests: normal, edge and adversarial

Graders

Deterministic checks first (schema, enum, citations); LLM-as-judge with a rubric for the rest

CI gate

pytest + promptfoo run on every PR; merge blocked below thresholds

Metric	How measured	Gate
Task success	Rubric score on golden set	**≥ 90%**
Tool-call accuracy	Right tool, valid args	**≥ 95%**
Groundedness	Citations match sources	**≥ 90%**
Injection block rate	Attack cases stopped	**≥ 95%**
PII leakage	Scanner on all outputs	**0**
p95 latency · cost	Per request, from logs	**Agreed SLO**

Thresholds are proposals; pods agree final values in the spec.

Red-Team the Day 1 App: Pods Swap and Attack

Each pod gets another pod's tagged build and a library of attack scripts. Every success is logged with a reproducible script.

Direct prompt injection

LLM01 · ASI01

Indirect injection via ticket or SOP

LLM01 · ASI01

Tool misuse: empty or broad filter

LLM06 · ASI02

PII and secret exfiltration

LLM02 · LLM07

MCP tool-description poisoning

ASI04

Knowledge-base poisoning

LLM04 · ASI06

Rubber-stamp the human gate

ASI09

Loop and cost exhaustion

LLM10 · ASI08

Tooling

promptfoo red-team

Plugin-based attack generation against the API

garak

LLM vulnerability probes

PyRIT

Multi-turn, scripted attack orchestration

redteam/ pytest suite

UNext scripts targeting MCP tools and handoffs

Harden, Re-Test, Score — Then Show It Live



Showcase: each pod in 5 minutes

- Spec in → agents running on Bedrock
- The attack that worked against them
- The guardrail layer that now stops it, with the audit log
- Scorecard before and after hardening

Resilience scorecard	Target
Breaks fixed and re-tested	100%
Breaks added as regression cases	100%
Unauthorised tool calls	0
PII / secret leakage	0
Eval harness pass rate	≥ 90%
Audit log completeness	100%

What We Need in Place by Tuesday 3 November

Trainer access at least one working day before Day 1 so every lab is run end to end in the Philips environment.

Philips MR team provides

- AWS account access for trainer and participants (SSO / IAM role)
- Bedrock model access enabled in the approved region
- Permissions to create and apply Bedrock Guardrails
- OpenAI / Claude API endpoints and keys for Copilot / Codex
- Network allowlist: PyPI, GitHub, AWS endpoints
- Use-case confirmation or replacement by 30 Oct

UNext provides

- Starter repo with env_check, scaffold and fallback branches
- Spec, plan and task templates
- Skills: scaffolding, MCP, guardrails, red-team
- Synthetic MR service dataset and SOP corpus
- Eval golden set, attack scripts and scorecard
- Trainer on site for both days

Participants bring

- Laptop with admin rights to install packages
- Prerequisites installed per the setup guide
- Env check run once before 4 Nov
- Copilot or Codex licence active
- Basic Python and Git fluency

Thank You

unext